

Documento D3 - ReValue

1. Introduzione

ReValue e una piattaforma per favorire il riuso di oggetti tra utenti. Il donatore pubblica un annuncio con descrizione, categoria, scadenza e posizione; l'acquirente consulta il catalogo, prenota l'oggetto e completa lo scambio tramite QR Code. Il sistema assegna punti wallet a scambio concluso e mantiene controlli su prenotazioni, ritiri, messaggi e segnalazioni.

1.1 Recap funzionale

- Catalogo pubblico con filtri per categoria, dimensione, materiale, scadenza e distanza.
- Registrazione e login con JWT.
- Gestione annunci con creazione, modifica, cancellazione logica e stati.
- Prenotazioni con protezione da doppia prenotazione concorrente.
- Scambio tramite QR Code generato dal donatore e validato dall'acquirente.
- Wallet con saldo e storico transazioni.
- Chat/messaggi tra partecipanti allo scambio.
- Funzioni admin per segnalazioni, utenti e annunci.

1.2 Stack

Area	Tecnologia
Backend	Node.js, Express 5
Database	MongoDB con Mongoose
Autenticazione	JWT, bcrypt
Test	Jest, Supertest, mongodb-memory-server
Frontend web	HTML, CSS, JavaScript statico
Mobile	React Native / Expo
Logging	Morgan

1.3 Team

Nome	Matricola	Focus principale
Alessandro Gremes	242330	Backend, auth, annunci, documentazione D3
Paolo Sarcletti	246846	QR, prenotazioni, test E2E
Alessandro Turri	244927	Frontend/mobile, catalogo, profilo

2. Architettura Tecnica

2.1 Struttura cartelle

```
ReValue/  
  backend/  
    app.js  
    server.js  
  src/  
    controllers/  
    middleware/
```

```
models/
routes/
services/
utils/
tests/
  integration/
  unit/
frontend/
  index.html
  js/
  views/
mobile/
  App.js
  src/
    api/
    components/
    screens/
    theme/
docs/
  D3.md
```

2.2 Flusso backend

```
Client web/mobile
-> Express app
-> middleware globali (CORS, body parser, morgan)
-> routes /api/*
-> middleware di sicurezza (authenticate, notSospeso, requireAdmin)
-> controller
-> service quando presente
-> model Mongoose
-> MongoDB
```

I controller restano responsabili delle risposte HTTP. La logica auth è stata estratta in `backend/src/services/authService.js`; wallet e chat hanno già service dedicati. Gli errori non intercettati e le route API inesistenti passano da `backend/src/middleware/errorHandler.js`, che restituisce il formato:

```
{
  "ok": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Route GET /api/example non trovata"
  }
}
```

2.3 Moduli principali

Modulo	Responsabilit�
auth	Registrazione, login, logout, JWT
users	Profilo utente e profilo pubblico
annunci	Catalogo, CRUD annunci, stato annuncio
prenotazioni	Creazione, annullamento, no-show, disdetta

qr / scambi	Generazione e validazione QR Code
wallet	Saldo, storico, accredito e sottrazione punti
messaggi / conversazioni	Storico chat, invio messaggi, non letti
admin	Statistiche, segnalazioni, ban/sospensione utenti, moderazione annunci

3. Sprint 1 Backlog

Persona	Task	Stato
Alessandro Gremes	Bug fix auth/users/annunci: token scaduti e validazioni annunci	Completato
Alessandro Gremes	Refactor logica auth in services/authService.js	Completato
Alessandro Gremes	Middleware errorHandler.js per errori standardizzati	Completato
Alessandro Gremes	Logging richieste con Morgan	Completato
Paolo Sarcletti	Bug fix QR/prenotazioni	Completato
Paolo Sarcletti	Test E2E scambio completo	Completato
Alessandro Turri	Catalogo web/mobile e profili	Completato
Team	Documento D3 e report test	Completato

4. Endpoint API

4.1 Auth

Metodo	Endpoint	Auth	Descrizione
POST	/api/auth/register	No	Registra utente e crea wallet
POST	/api/auth/login	No	Effettua login e restituisce JWT
POST	/api/auth/logout	Si	Logout logico lato client

4.2 Users

Metodo	Endpoint	Auth	Descrizione
PUT	/api/users/me	Si	Aggiorna profilo utente
GET	/api/users/:id/profilo	No	Legge profilo pubblico

4.3 Annunci

Metodo	Endpoint	Auth	Descrizione
GET	/api/annunci	Opzionale	Catalogo pubblico con filtri e paginazione
GET	/api/annunci/me	Si	Annunci dell'utente autenticato
GET	/api/annunci/:id	Opzionale	Dettaglio annuncio
POST	/api/annunci	Si	Crea annuncio
PUT	/api/annunci/:id	Si	Modifica annuncio

DELETE	/api/annunci/:id	Si	Cancella logicamente annuncio
PATCH	/api/annunci/:id/stato	Si	Cambia stato annuncio

4.4 Prenotazioni

Metodo	Endpoint	Auth	Descrizione
POST	/api/prenotazioni	Si	Crea prenotazione
GET	/api/prenotazioni/me	Si	Lista prenotazioni dell'utente
GET	/api/prenotazioni/:id	Si	Dettaglio prenotazione
DELETE	/api/prenotazioni/:id	Si	Annula prenotazione
POST	/api/prenotazioni/:id/no-show	Si	Segnala mancato ritiro
POST	/api/prenotazioni/:id/disdici	Si	Disdice ritiro

4.5 QR e Scambi

Metodo	Endpoint	Auth	Descrizione
POST	/api/qr/genera	Si	Genera QR per prenotazione
POST	/api/qr/valida	Si	Valida QR e completa scambio
GET	/api/scambi/:prenotazioneId/qr	Si	Recupera QR esistente
POST	/api/scambi/:prenotazioneId/valida	Si	Valida scambio tramite codice QR

4.6 Wallet

Metodo	Endpoint	Auth	Descrizione
GET	/api/wallet/me	Si	Saldo e storico in una risposta
GET	/api/wallet/saldo	Si	Solo saldo
GET	/api/wallet/storico	Si	Storico paginato e filtrabile

4.7 Messaggi e Conversazioni

Metodo	Endpoint	Auth	Descrizione
GET	/api/messaggi/:prenotazioneId	Si	Storico messaggi per prenotazione
POST	/api/messaggi/:prenotazioneId	Si	Invia messaggio
PATCH	/api/messaggi/:id/letto	Si	Marca messaggio come letto
GET	/api/conversazioni/me	Si	Lista conversazioni utente
GET	/api/conversazioni/me/non-letti	Si	Conteggio messaggi non letti
GET	/api/conversazioni/:id/messaggi	Si	Storico paginato conversazione
POST	/api/conversazioni/:id/messaggi	Si	Invia messaggio in conversazione

4.8 Admin

Metodo	Endpoint	Auth	Descrizione
GET	/api/admin/statistiche	Admin	Dashboard statistiche
GET	/api/admin/segnalazioni	Admin	Lista segnalazioni
POST	/api/admin/utenti/:id/ban	Admin	Banna utente
POST	/api/admin/utenti/:id/sospendi	Admin	Sospende utente
POST	/api/admin/utenti/:id/riabilita	Admin	Riabilita utente
PATCH	/api/admin/annunci/:id/forza	Admin	Forza stato annuncio
DELETE	/api/admin/annunci/:id	Admin	Rimuove annuncio

5. Report Test

5.1 Comando eseguito

```
cd backend
npm test -- --runInBand
```

5.2 Esito

Esecuzione del 13 maggio 2026:

Suite	Esito
tests/integration/qr_edge_cases.test.js	PASS
tests/integration/concurrent.test.js	PASS
tests/integration/swap.test.js	PASS
tests/integration/catalog_flow.test.js	PASS
tests/unit/auth.test.js	PASS
tests/unit/annunci.test.js	PASS
tests/unit/wallet.test.js	PASS
tests/unit/prenotazioni.test.js	PASS

Totale: 8 suite passate, 72 test passati, 0 falliti.

5.3 Copertura funzionale verificata

- Auth: registrazione, login, email duplicata, password errata, token JWT scaduto.
- Annunci: CRUD modello, soft delete, filtri, ordinamento, optimistic locking.
- Prenotazioni: modello e concorrenza su doppia prenotazione.
- QR: prenotazioni completate/annullate, token scaduto, validazione scambio.
- Flusso E2E: registrazione, creazione annuncio, prenotazione, QR, wallet.
- Wallet: creazione, accredito, sottrazione, saldo insufficiente, storico.

6. Note Finali

- Il logging HTTP è attivo con `morgan('combined')` in produzione e `morgan('dev')` in sviluppo.
- Le route API non trovate restituiscono un errore standardizzato.

- La logica auth è isolata nel service layer per rendere i controller più sottili e testabili.
- La validazione JWT gestisce esplicitamente token scaduti, token invalidi e segreto mancante.